



Architecture Diagrams

Electric Vehicle Charging Station Management System



Electric Vehicle Charging Station Management System: Architecture Diagrams

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Home **i**

Electric Vehicle Charging Station Management System Diagram (Option 1) 1

Electric Vehicle Charging Station Management System Diagram (Option 2) 2

Electric Vehicle Charging Station Management System with AWS IoT Diagram 3

Download editable diagram 4

Create a free AWS account 5

Further reading 5

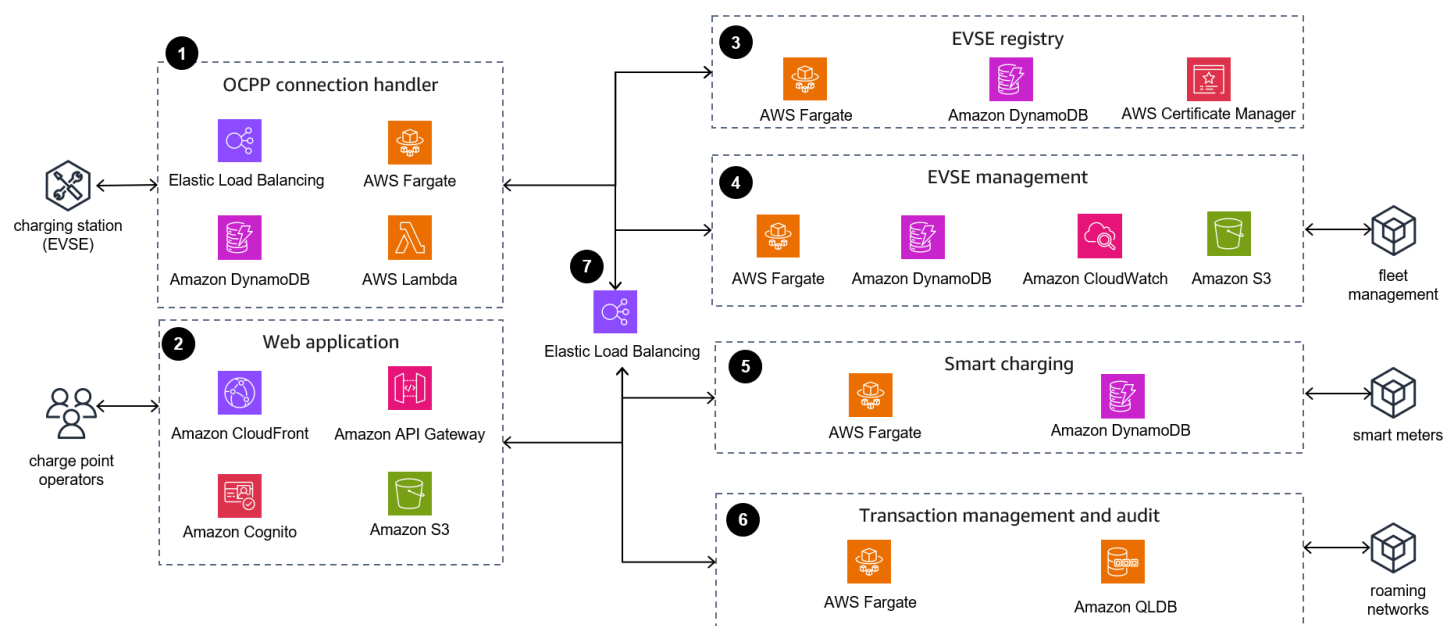
Diagram history 5

Electric Vehicle Charging Station Management System

Publication date: June 4, 2024 ([Diagram history](#))

Electric Vehicle Charging Station Management System Diagram (Option 1)

Container-based microservices architecture utilizing familiar technologies and concepts, most suitable for organizations embarking on containerization. Whether you're planning a development from scratch or the modernization of existing product, this architecture provides a scalable, reliable, and cost-efficient solution for the critical electric vehicle (EV) infrastructure.

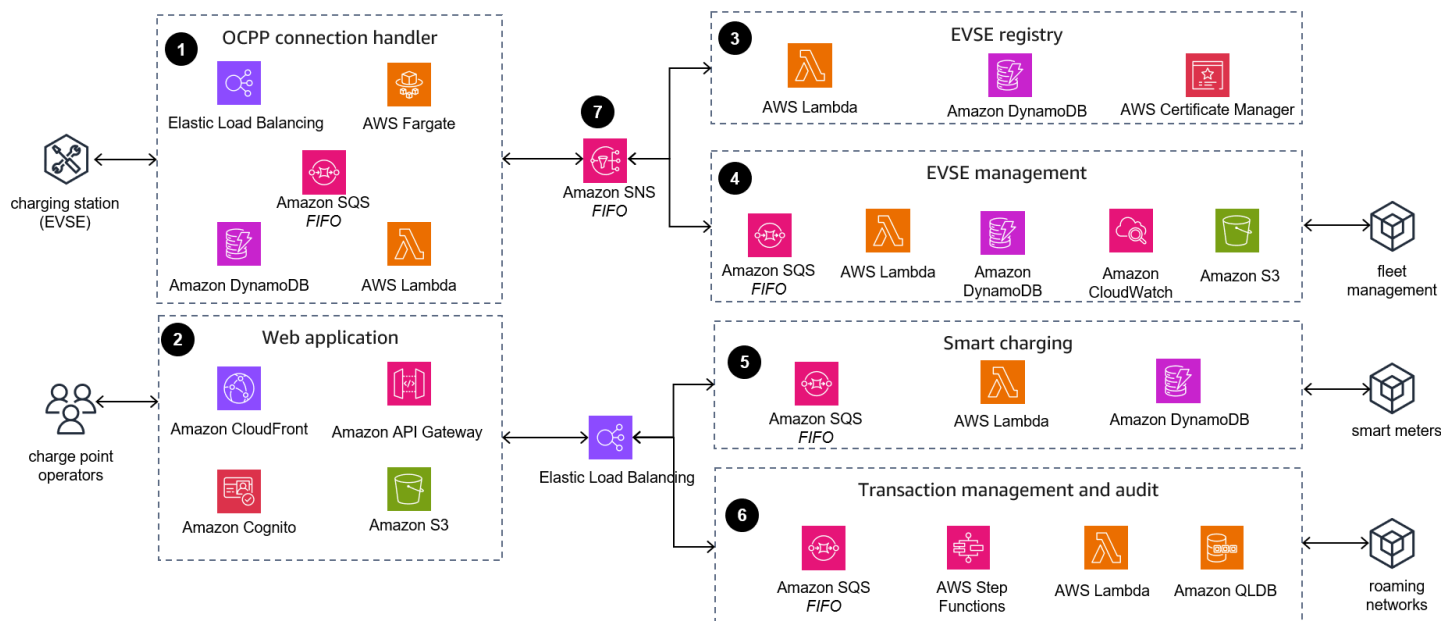


1. Charging stations (electric vehicle supply equipment, or EVSE) are connected to **Amazon Elastic Container Service (Amazon ECS)** on **AWS Fargate** behind **Network Load Balancer**. **AWS Lambda** routes outbound open charge point protocol (OCPP) messages to EVSEs and **Amazon DynamoDB** keeps active connections.
2. **Amazon CloudFront** serves static EVSE management web application from an **Amazon Simple Storage Service (Amazon S3)** bucket. **Amazon API Gateway** exposes the backend REST API for the web application. **Amazon Cognito** stores the Charging Point Operator's user identities.
3. **DynamoDB** stores the registry of EVSE with credentials in encrypted form. **AWS Certificate Manager (ACM)** manages EVSE certificates for authentication.

4. Management microservice delivers EVSE metrics to and from **Amazon CloudWatch**. **Amazon S3** is used to store EVSE logs, cold metrics, and firmware files.
5. The configuration of charging sites, balancing algorithms, and power grid capacity stored in **DynamoDB**.
6. **Amazon Quantum Ledger Database (Amazon QLDB)** stores immutable and cryptographically verifiable log of charging transactions.
7. **Application Load Balancer** exposes the internal APIs of the microservices, routes requests, and balances between multiple tasks on **Amazon ECS**.

Electric Vehicle Charging Station Management System Diagram (Option 2)

Event-driven serverless architecture most suitable for organizations with existing know-how in serverless technologies and mature DevOps culture. It provides a high degree of scalability, fault isolation, and minimal running costs when not in use.

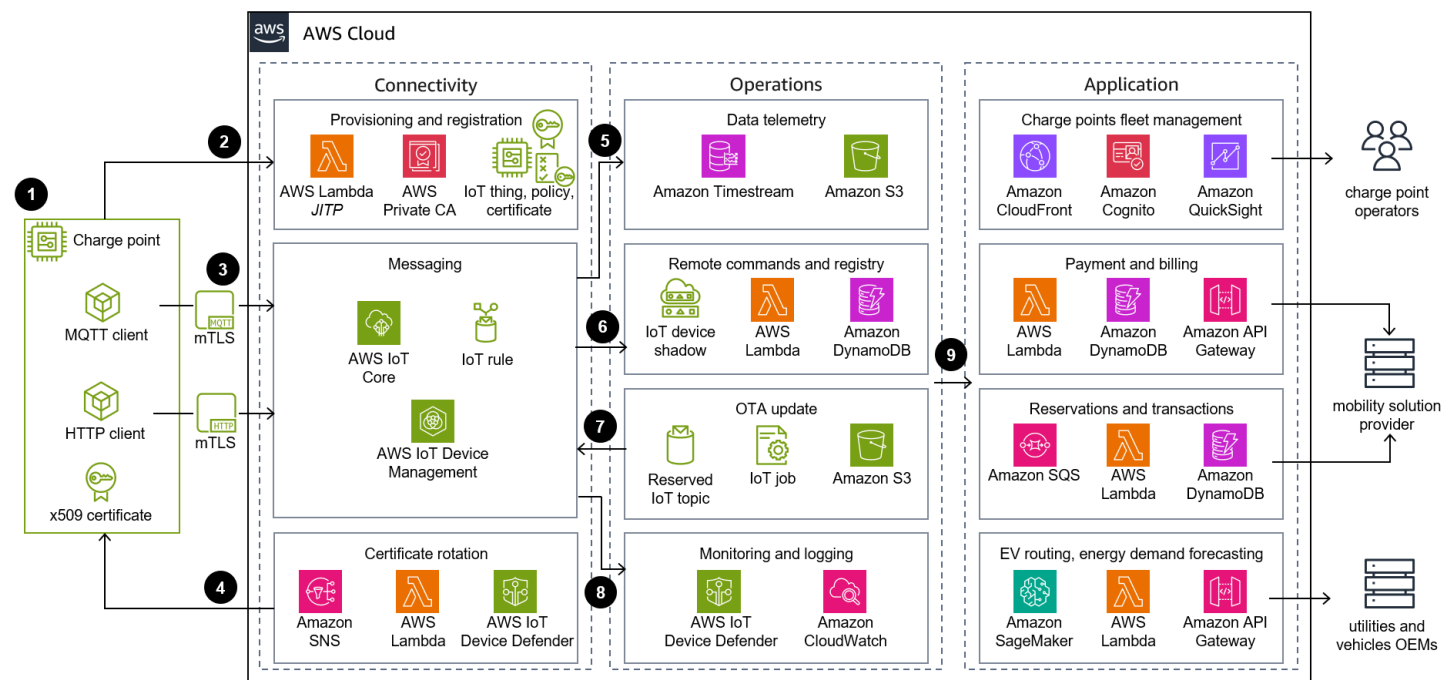


1. EVSEs are connected to **Amazon ECS** on **Fargate** behind **Network Load Balancer**. **Lambda** routes outbound OCPP messages to EVSEs and **DynamoDB** keeps active connections.
2. **CloudFront** serves static EVSE management web application from an **Amazon S3** bucket. **API Gateway** exposes the backend REST API for the web application. **Amazon Cognito** stores the charging point operator's user identities.

3. **DynamoDB** stores the registry of EVSE with credentials in encrypted form. **ACM** manages EVSE certificates for authentication.
4. **Lambda** functions deliver EVSE metrics to and from **CloudWatch**. **Amazon S3** used to store EVSE logs, cold metrics, and firmware files.
5. The configuration of charging sites, balancing algorithms, and power grid capacity stored in **DynamoDB**.
6. **AWS Step Functions** orchestrates the transaction management workflows. **Amazon QLDB** stores immutable and cryptographically verifiable log of charging transactions.
7. **Amazon Simple Notification Service** (Amazon SNS) first in, first out (FIFO) topics transport messages to and from EVSEs. Messages are fanned out to **Amazon Simple Queue Service** (Amazon SQS) queues before being processed by **Lambda** functions.
8. **Application Load Balancer** constructs an internal APIs of **Lambda**-based microservices.

Electric Vehicle Charging Station Management System with AWS IoT Diagram

This reference architecture demonstrates how to build a highly-scalable, low-latency electric vehicle (EV) charge point operator system based on the EV industry standard, Open Charge Point Protocol (OCPP), using AWS services like AWS IoT Core and AWS Lambda.



1. An EV charge point is deployed in the field, either as a domestic charge point or as a fast charging one. It is fitted with both sensors and actuators (to stop and start charging) and can connect to the internet.
2. The EV charge point is provisioned as an **AWS IoT** thing using an **AWS Private Certificate Authority** (AWS Private CA) x509 certificate, along with the relevant **AWS IoT** policy, using just-in-time provisioning (JITP) with **AWS Lambda**.
3. The EV charge point connects to AWS using the MQTT protocol through **AWS IoT Core**, acting as the entry point to every message sent from and to the cloud. **AWS IoT** rules are initiated and sent to relevant AWS services and applications.
4. Following security best practices and guidance, the EV charge point certificate is rotated on a regular basis, leveraging **AWS IoT Device Defender** to identify certificates close to expiration, **Amazon Simple Notification Service** (Amazon SNS) and **Lambda** to handle the new certificate generation and rotation.
5. Telemetry data coming from the EV charge point through **AWS IoT Core** using **AWS IoT** rules and actions is stored into a dedicated storage service such as **Amazon Timestream** or **Amazon Simple Storage Service** (Amazon S3) for further analysis and fleet management purposes.
6. An **AWS IoT** device shadow can make a charge point state available to the application layer and other services whether the device is connected to **AWS IoT Core** or not, implementing the preferred mechanism for IoT device remote commands.
7. Use **AWS IoT Device Management** to implement over-the-air (OTA) update management through IoT jobs, allowing the EV charge point to download its new firmware from **Amazon S3**.
8. Get security alerts from **AWS IoT Device Defender** and analyze IoT logs in **Amazon CloudWatch**. Use **AWS IoT** fleet indexing to manage state, connectivity, and device violations and to organize, investigate, and troubleshoot your fleet of devices.
9. Build your business logic applications serving different user personas from fleet management operators to utilities, mobility solution providers or vehicle OEMs. Use AWS services such as **Amazon SageMaker AI** for energy demand forecasting or EV routing algorithms, **Amazon Quick** to build fleet management dashboards, or **Amazon API Gateway** to provide API access to your data to third parties, partners, and customers.

Download editable diagram

To customize this reference architecture diagram based on your business needs, [download the ZIP file](#) which contains an editable PowerPoint.

Create a free AWS account

[Sign up now](#)

Sign up for an AWS account. New accounts include 12 months of [AWS Free Tier](#) access, including the use of Amazon EC2, Amazon S3, and Amazon DynamoDB.

Further reading

For additional information, refer to

- [AWS Architecture Icons](#)
- [AWS Architecture Center](#)
- [AWS Well-Architected](#)

Diagram history

To be notified about updates to this reference architecture diagram, subscribe to the RSS feed.

Change	Description	Date
Diagram updated	AWS IoT option added.	June 4, 2024
Initial publication	Diagram first published	January 3, 2022

Note

To subscribe to RSS updates, you must have an RSS plugin enabled for the browser you are using.